THE UNIVERSITY OF HONG KONG

FACULTY OF ENGINEERING
DEPARTMENT OF COMPUTER SCIENCE

COMP7104/DASC7104 - Advanced Database Systems

Date: December 14, 2022 **Time: 6:30pm-8:30pm**

<INSTRUCTION ABOUT NUMBER OF QUESTION TO BE ANSWERED: OVERALL AND FROM EACH SECTION>
35 questions overall, divided into two sections as follows:
Section 1 – Multiple-choice questions – 25 questions
Section 2 – Detailed-answer questions – 10 questions

<INDICATION OF COMPULSORY QUESTIONS>
None.

<INFORMATION ABOUT THE MARK VALUE OF QUESTIONS>
Section 1 – 1 point per question – 25/100 points in total
Section 2 – number of points depends on the question – 75/100 points in total

<INDICATION OF REQUIREMENT TO ANSWER QUESTIONS OR SECTIONS IN SEPARATE ANSWER BOOKS>
None

<MATERIAL PROVIDED>
None

<RUBRIC CONCERNING THE USE OF ELECTRONIC CALCULATORS (if necessary)>
Only approved calculators as announced by the Examinations Secretary can be used in this examination. It is candidates' responsibility to ensure their calculator operates satisfactorily, and candidates must record the name and type of the calculator used on the front page of the examination script.

General indications

- 1 KB = 1024 bytes; i.e., we will be using powers of 2, not powers of 10
- For questions involving arithmetic expressions, numerical work or derivations, credit will be given only if the work leading to the answer is shown.

Section 1 – Multiple-choice questions – 25 questions – 1 point per question

Circle the true answer(s) to each question. Each question may have zero, one, or several valid answers. For each question, only the identification of all and only the valid answers will allow you to obtain the assigned point.

Q1.1 Which of the following assertions about storage devices is / are true?

A) Flash (SSD) memory is preferable to spinning disk (HDD) because reads are more predictable.

B) When reading data, locality matters for both spinning disk and flash (SSD).

C) Writing to a flash (SSD) device is less costly than reading from a flash (SSD) device.

Explanation: A is true (due to the intrinsic design of SSD, reads are more predictable, as there is no difference between random and sequential reads); B is false, locality for reads is not important in SSDs, as there is no difference between random and sequential reads; C is false: wear leveling and coarser grained writes make writing more costly than reads.

Q1.2 Which of the following assertions about storage devices and B+-tree indexing is / are true?

A) When indexing data that is mostly accessed by read operations, SSDs are preferable to HDD.

B) In a bulk-loaded B+-tree index, as data pages (leaf nodes) will be stored sequentially on disk, we can save space by not having next / previous pointers between them.

C) Assuming that the indexing parameter that is the B+-tree node size can be a multiple of page size on disk, this parameter should be adapted to the disk's access latency.

Explanation: A is true, as random reads are as fast as sequential reads, which makes index accesses at any level equally fast. B is false: we must still keep such pointers, since the index is dynamic and the initial sequential layout may not be guaranteed later on. C is true, since larger nodes will mean fewer accesses.

Q1.3 Which of the following assertions about storage devices is / are true?

A) Among RAM, SSD, and HDD, only one storage medium is block-addressable.

B) When querying for a 8KB record, exactly 8KB of data is read from disk.

C) In an HDD, a random access is one that does not take into account the position of read heads.

Explanation: A is false, as both SDD and spinning disk are block-addressable (as opposed to byte addressable B is false, we read a block of data, which is usually more than 8KB of data. C is true: a random read may happen anywhere, regardless of the current position of the access heads.

Q1.4 Which of the following assertions about paginated database files (files-of-pages) is / are true?

A) Log-structured files are not adapted for read-mostly data.

B) In a file that is close to full (very few pages have very little free space), organized as a page-directory, the performance of inserting a new record degrades to the one of doing a full scan over the data pages.

C) A disadvantage of the chained-lists implementation compared to the page-directory one is the need to do a look-up in a catalog, before accessing the database file.

Explanation: A true – indeed reading the data may require "replaying" some log entries; for appends / modification / deletes the log-structured file is faster. B is false, we will mostly likely need to scan a large portion or the entire level of the directory pages (which is much less than the data pages). C is false, the same (entry point, master page) kind of information must be kept in a catalog even for the page-directory organization.

Q1.5 Which of the following assertions about database pages (pages-of-records) is / are true?

A) When dealing with fixed-length records without nullable fields by an unpacked encoding, the size of the bitmap may vary from one page to another in the same file.

B) Fragmentation may be an issue for slotted pages of variable-length records, but dealing with it does not require changing recordIds.

C) For fixed-length records, a slot directory has worst space complexity than a bitmap.

Explanation: A is false, since the size of the records is fixed, so the number of records we can fit on a page (hence the bitmap size) is also fixed. B is true, even if we reorganize the data the Ids do not change. C is true, bitmaps have better space complexity since the slot directory uses pointers.

Q1.6 Which of the following assertions is / are true about the benefits of using a record footer for variable length records (records-of-attributes)?

A) Does not need a delimiter character to separate fields in the records.

B) Is always better or as good as the fixed-length record format.

C) Can access any field without scanning the entire record.

D) Has compact representation of null values.

Explanation: all false, a record header (not footer) is used.

Q1.7 How many Alternative 3 B+-tree indexes can be created on a table?

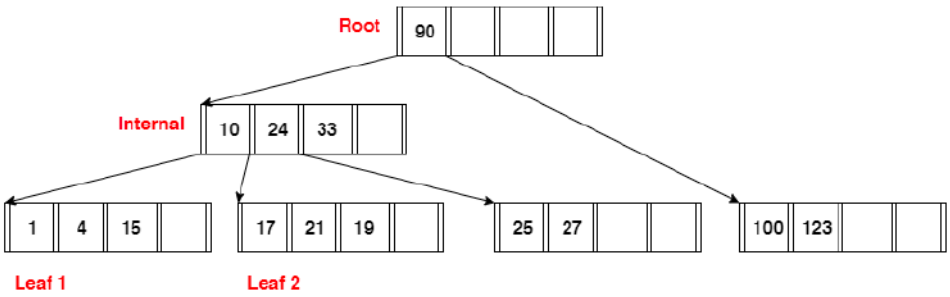
A) At most 1.

B) Exactly 1.

C) As many as we want.

Explanation: the data is in the index file, so unless we want to replicate it, only one such index can be built.

Q1.8 Which of the following statements is / are true about the B+-tree below?



A) The tree is invalid because at least one leaf node violates the key order property.

B) The tree is invalid because the search invariant property does not hold at the internal node.

C) The tree is balanced but its height can be decremented if we reorganize the data (e.g., by bulk loading in a new index).

Explanation: A is true, since the second leaf does not have ordered entries. B is true (since entry 10 less than 15), C is false, since the tree is not balanced.

Q1.9 Which of the following statements is / are true about doing interval searches in a unclustered B+-tree?

A) It may cause as many random (non-sequential) reads of blocks in the data file as there are matching records for the search condition.

B) If for most queries few (say less than 1%) records match the search condition, an unclustered index will be preferable to a clustered one.

C) If the index is stored on an SSD, the unclustered index will always be better than a clustered one.

Explanation: A is true, since the index is unclustered. B is true since the clustered index comes with overhead (keeping the data almost ordered). C is false, since in some cases (many matches) the clustered index can do better, even with the ordering overhead, simply because it will do fewer accesses (even if there is no difference in cost between random and sequential accesses in an SSD).

Q1.10 Which of the following assertions about buffer management in DBMSs is / are true?

A) LRU cannot prevent sequential flooding during sequential scans.

B) We use the clock replacement policy over LRU in order to run faster the replacement policy.

C) MRU is as expensive to run as LRU.

Explanation: A is true (as seen in class), B is true (efficiency is the reason for introducing clock), C is true (maintaining a min-queue or a max-queue has the same complexity.

Q1.11 Which of the following assertions about out-of-core sorting / hashing is / are true?

A) To finish the final pass of external hashing (conquer), we use a fine-grained hash function, h_r, that should be different from the hash functions used in the previous passes.

B) De-duplicating a file can be done by either sorting or hashing, depending on the context.

C) Given a hashed file, there is some permutation of pages such that the resulting file is sorted.

Explanation: A is true since to finish the final pass of external hashing (conquer), we use a fine-grained hash function, hr. This is the reason why the full partition must fit in memory, and it's the only way we guarantee that when a partition exits the conquer pass, it is hashed. B is true, since the file could already be in close to sorted order. C is false: consider a file with a single page that can fit 3 records: C, A, B. This file is hashed, but there is no permutation of the 1 page that would sort it.

Q1.12 We have a table with five million pages (N = 5,000,000 pages) and we want to sort it using external merge sort (out-of-core sorting). Assume that the DBMS is not using double buffering. Let B=6 denote the number of available buffer frames. Which of the following assertions is / are true about out-of-core sorting?

A) We need 10 passes to sort the data.

B) Even with 5 times less data (N = 1,000,000 pages), we would still need as many passes to sort it than for N = 5,000,000 pages.

C) We need roughly 20 times more buffers to sort the data in just two passes.

D) We need roughly 30 times more buffers to sort the data in just three passes.

Explanation: A) is true, since 1 + ceil($\log_{B-1}$(ceil(N/B))) = 1 + ceil($\log_5$(833334)) = 10; same formula applies to 1 million, so B) is also true. C) is false, while D) is true: (B x $(B-1)^2$ must be more than 5 million, so B can be 172, so roughly 30 times more.

Q1.13 Which of the following assertions is / are true about out-of-core sorting / hashing?

A) Increasing the number of buffer pages does not affect the number of I/Os performed in Pass 0 of an external hashing.

B) When sorting, double-buffering does not reduce I/O cost, but it reduces the overall execution time.

C) In the out-of-core sorting algorithm, for every pass except the last one, we keep 1 buffer frame reserved as the output buffer

Explanation: A is true: regardless of the number of buffer pages, every pass of an external sort / sort (including Pass 0) performs the same number of I/Os. B true: double buffering allows a program to concurrently perform computations and fetch data from disk, so to have the CPU and the disk work in parallel. C is false, we keep an output buffer in all passes except the first one.

Q1.14 Which of the following assertions about join algorithms is / are true?

A. In CNLJ (Chunk Nested Loop Join) the DBMS has to repeatedly do sequential scans to check for a matches in the outer table.

B. In INLJ (Index Nested Loop Join) the inner table is the one with an index.

C. We cannot apply INLJ unless an index is available on the join attribute of either one of the joined tables *before* we compute the join.

D. If we are joining two tables that are very different in size, if both relations have indexes on the join attribute, with INLJ it is better to query the index of the larger relation.

Explanation: A is false: we repeatedly scan the inner table; B is true; C is false – we can always come up with examples where it may be cheaper to just build an index from scratch just for the join. D is true: consider the cost model for INLJ, where L = larger and S = smaller. If we query S, the cost is [L] + size(L)x (cost to lookup in S), whereas if we query L the cost is [S] + size(S) x (cost to lookup in L). The cost to lookup in an index will not change much even if one table is much bigger but the number of records can change drastically, so you want to lookup in the larger table.

Q1.15 Which of the following assertions about join algorithms is / are true?

A) SMJ (Sort-Merge Join) should be preferred to CNLJ if one of the tables has a clustered index on the join attribute.

B) The best-case scenario of GHJ (Grace Hash Join) is when the join attribute for all the records in both tables contain the same value.

C) SMJ should be preferred to GHJ for queries over unsorted, non-uniform data.

Explanation: A is true, since a clustered index means one of the tables is already sorted. B is false, as this is the worst-case scenario. C is true since non-uniformed (skewed) data means the hashing may be leading to large partitions.

Q1.16 Which of the following **is / are not** an interesting optimization choice over a logical query plan:

A) Apply filters as early as possible (predicate pushdown).

B) Reorder predicates so that they are applied in reverse order with respect to their selectivity (sel), i.e., the one with the highest selectivity first.

C) Split a complex, conjunctive predicate and push it down.

D) Project away unnecessary columns before joining two tables.

Explanation: A, C, D are all good choices, B is not since it should in order (one with lowest selectivity first). Hence B is the only valid answer.

Q1.17 Which of the following assertions about the choice of the System R / Selinger query optimizer approach of only looking at left-deep join trees is / are true?

A) It allows us to never consider plans with cartesian products.

B) If given a perfect cost estimator, it will always find the optimal plan.

C) The running time of the optimization algorithm is polynomial in the number of tables.

D) Left-deep join trees allow us to consider the plans in which the DBMS does not need to materialize the outputs of join operators.

Explanation: A is false, as the query may be performing a cartesian product; B is false, as the optimal plan may not be left-deep; C is false, as the complexity is exponential still in the number of tables; D is true, as we get to see all pipelined plans.

Q1.18 Of the following queries, which one(s) require a blocking operator in their execution plan ?

A) Select title from Book ;

B) Select distinct title from Book ;

C) Select title from Book where year > 2020 ;

D) Select title from Book order by year ;

Explanation : A and C do not need operators that require to see all the input before producing output ; B and D must sort / group in order to eliminate duplicates or display order, hence require a blocking operator.

Q1.19 Which of the following assertions is / are true about transactions and the strict 2PL protocol?

A) Schedules generated by strict 2PL cannot have dirty reads.

B) Schedules generated by strict 2PL cannot cause deadlocks.

C) A conflict serializable schedule will never contain a cycle in its precedence graph.

D) Every serializable schedule is conflict serializable.

Explanation: A and C are true by the definition of conflict serializability, B is false as deadlocks may still arise, D is false as the inclusion is in the other direction.

Q1.20 Which of the following properties is / are true about locking-based concurrency control?

A) Transactions request locks (or upgrades) from the lock manager.

B) Locking-based concurrency control is a form of optimistic concurrency control

C) The lock-table must be durable, in order to retrieve the locks of a transactions that were active (i.e., still running) at the moment of a crash.

D) The lock manager ensures that all transactions get a fair access to locks.

Explanation: A is true (by definition of locking mechanisms); B is false (it is pessimistic); C is false, as transactions still active (i.e., still running) when the DBMS crashes are automatically aborted ; D is false as there is no logic of fairness.

Q.21 Which following assertions is / are true about FORCE/NO-FORCE and STEAL/NO-STEAL policies for Crash Recovery ?

A) With NO STEAL all the data that a transaction needs to modify must fit in memory.

B) WAL is the implementation of STEAL + NO-FORCE having the fastest recovery performance.

C) FORCE makes it easier to recover since all of the changes are in preserved but requires larger buffers.

D) The hardest buffer pool management policy to implement is NO-STEAL + FORCE.

Explanation: A is true (since pages remain in RAM), B is false since it does not have the fastest recovery (rather the opposite), C is false since FORCE results in poor runtime performance (many I/Os), but not needing more RAM. D is false (it is the easiest)

Q1.22 Which of the following assertions is / are true about PDBMS partitioning strategies?

A) With horizontal partitioning, we split a table's records into disjoint subsets of records of equal size.

B) Doing a key lookup over hash partitioned data requires fewer I/Os than if the data is partitioned round-robin partitioned.

C) Lookup by range over range partition data will always be better than over hash partitioned data.

D) Round-robin partitioning cannot lead to data skew.

Explanation: A is false, since we do not necessarily aim for equal size (unless round-robin); B is true, since round robin means scanning at all nodes; C is false (no such requirement), C is false, especially if the lookup is not on the partition key. D is true, by definition since nodes get equal shares of the data.

Q1.23 Which of the following assertions is / are true about the 2PC distributed commit protocol?

A) Distributed deadlock occurs only if the waits-for graph at a node forms a cycle.

B) In 2PC, we need all participant nodes to agree on abort (vote NO) to abort a transaction.

C) In 2PC phase 2, after the coordinator sends commit message to all participants, participants respond with Ack after they generate and flush the commit record.

D) A failed participant to a distributed transaction T can cause successful participants to T to abort.

Explanation: A is false: distributed deadlock occurs whenever the union of waits-for graphs across different nodes forms a cycle. B is false: unanimity is only required for commit. C is true:participants respond with Ack after generating and flushing the commit record. D is true since unanimity is required.

Q1.24 Which of the following assertions is / are true about partitioning and replication in NoSQL databases?

A) Both can improve scalability.

B) Both can reduce the query execution latency.

C) Both can handle ever-increasing database sizes.

D) We can have partitioning without replication, but we cannot have replication without partitioning.

Explanation: A. is true, even if one improves data scalability, the other workload scalability; B is true, with partitioning we can run queries faster, with replication we can answer queries with close by data in a geo-replicated scenario; C is false (only partitioning helps with data scaling). D is false. We can replicate at the granularity of entire databases or tables without partitioning them.

Q1.25 Which of the following NoSQL aspects are illustrated by the Apache Cassandra system?

A) Hash partitioning.

B) Schema-on-read mechanisms.

C) Horizontal scaling.

D) Synchronous propagation of updates to replicas.

Explanation: A and C are true: data is partitioned by hashing and we can distribute both the data and the load over many nodes, B is false (schema is required), D is true (tunable consistency in the range eventual-strong consistency).

Section 2 – Detailed-answer questions – 10 questions – 75 points in total

Q2.1 (Two-Phase Commit – 2PC) (4.5 points) Consider a distributed transaction T operating under the 2PC protocol. Let N0 be the coordinator node, and let N1, N2, N3 be the participant nodes. The following messages have been sent, in order of time:

time 1: N0 to N1: "Phase1:PREPARE"

time 2: N1 to N0: "OK"

time 3: N0 to N2: "Phase1:PREPARE"

time 4: N0 to N3: "Phase1:PREPARE"

(a) (1.5 points) Who should send a message next at time 5 and to whom?

(b) (1.5 points) Suppose that N0 received the "ABORT" response from N2 at time 5; what should happen next?

(c) (1.5 points) Suppose that N0 successfully receives all the "OK" messages from the participants from the first phase. It then sends the "Phase2: COMMIT" message to all the participants but N1crashes before it receives this message. What is the status of transaction T when N1 comes back online?

Explanation: A) Both N2 and N3 have to send a response to N0; B) The coordinator will mark the transaction as aborted, since unanimity is not met. C) T is committed: Once the coordinator (N0) gets a "OK" message from all participants, then the transaction is considered to be committed even though nodes may crash during the second phase.

Q2.2 (Join Algorithms) (11 points) Consider the relations R(x, y) and S(x, z, t) to be joined on the common attribute x. There are no indexes on these tables.

- We have B = 75 frames in the RAM buffer
- Pages are 5KB in size
- R has M =2400 pages with 80 records per page
- S has N = 1200 pages with 64 records per page

What is the **I/O cost** for the following joins algorithms, ignoring the costs of the final writing to disk of join results?

(a) (1 point) Chunk Nested Loop Join (CNLJ) with R as the outer relation and S as the inner relation.

(b) (1 point) Chunk Nested Loop Join (CNLJ) with S as the outer relation and R as the inner relation.

Let us consider next hash-join with S as the outer relation and R as the inner relation:

(c) (1 point) Under what assumptions can we ignore recursive partitioning?

(d) (1 point) Assuming no recursive partitioning is needed, detail the cost of the partitioning stage.

(e) (1 point) Assuming no recursive partitioning is needed, detail the cost of the probing (conquer) stage.

(f) (1 point) Assume that a large number of distinct x-values are hashed to the same of the buckets using the partitioning hash function h_p, leading to some partitions with more than B pages (say buckets 3, 10, and 74). What should we do?

Let us consider next sort-merge join with S as the outer relation and R as the inner relation:

(g) (1 point) What is the I/O cost of sorting the records in R on attribute x?

(h) (1 point) What is the I/O cost of sorting the records in S on attribute x?

(i) (1 point) What is the I/O cost of the merge phase assuming there are no duplicates in the join attribute?

(j) (1 point) What is the I/O cost of the merge phase in the worst case?

(k) (1 point) Assuming now that x is the primary key in both tables, and x and y are of the same type (say integers), what would be the maximal I/O cost of writing to disk the join results?

Explanation:

(a) M + ceil (M / (B-2)) x N = 42000 ;

(b) N + ceil (N / (B-2)) x M = 42000

(c) We must assume that hashing does spread rather evenly (uniformly) over the 74 buckets the x-values, such that no bucket gets more than 73 frames. We can make this assumption since 73x74=5402.

(d) 2 x (M + N) = 2 x (2400 + 1200) = 2 x 3600 = 7200

(e) (M + N) = (2400 + 1200) = 3600

(f) We must recursively hash buckets 3, 10 and 74 from both R and S, by some other hash function h_{p2}

(g) nb of passes = 1 + ceil($\log_{B-1}$ (ceil(M/B)))=2

So 2 x M x nb of passes = 2 x 2400 x 2 = 9600

(h) 2 x N x nb of passes = 2 x 1200 x 2 = 4800

(i) M + N = 2400 + 1200 = 3600

(j) M x N = 2400 x 1200 = 2880000

(k) x and y are each over 32 bytes, an S record is of size 80 bytes, a join record is over (size of S record) + 32 bytes, so 112 bytes; the size of the smaller relation S is 78000 records; so (76800 x 112) / (5 x 1024) = 1680

Q2.3 (Serializable transactions) (8 points) Consider the schedule given in the table below, where R and W stand for read and write respectively.

Timestep	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**
Transation1	R(A)					R(C)	R(B)		W(C)		
Transation2				R(C)						W(D)	W(E)
Transation3					W(A)			R(D)			
Transation4		R(E)	W(B)								

(a) (1 point) Is this schedule serial ? Why or why not ?

(b) (2.5 points) Draw the dependency graph of this schedule. Recall each edge in the dependency graph is of the form: Ti → Tj with an X label on the arrow indicating that there is a conflict on object X where Ti read/wrote on X before Tj.

(c) (1.5 points) Is this schedule conflict serializable? Why or why not ?

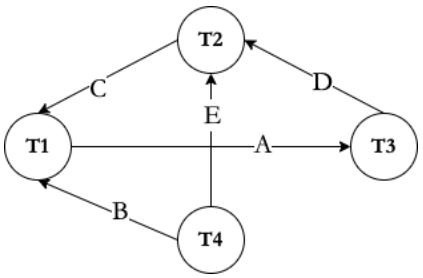
(d) (1.5 points) If your answer to (c) is 'yes', give a serial schedule to which our initial schedule is conflict equivalent. If the answer to (c) is 'no', indicate what removal of transaction(s) would make the initial schedule conflict serializable.

(e) (1.5 points) Is the initial schedule possible under (non-strict) 2PL ? Why or why not ?

Explanation:

a) NO, obviously since operations are interleaved in time;

b)



c) No, it is not conflict serializable since the graph has cycle T1-T3-T2

d) we could remove either one of T1, T2 or T3.

e) No, since 2PL gives us conflict serializable schedules.

Q2.4 (Deadlock detection) (5 points) Consider the following schedule of lock requests (X stands for exclusive, S stands for shared). Assume transactions never release locks in the considered time frame.

Timestep	1	2	3	4	5	6	7
Transaction 1			S(A)	S(B)			S(C)
Transaction 2	S(B)				X(A)		
Transation 3		X(C)				X(B)	
Lock Manager	g	g	g	g	b	b	b

(a) (1.5 points) For the lock requests in this table, determine which lock will be granted and which will blocked by the lock manager. Write directly in the table 'g' in the bottom row whenever the lock is granted and 'b' whenever the lock is blocked or the transaction has already been blocked by a former lock request. For example, in the table, the first lock (shared on B) at timestep 1 is marked as granted.

(b) (2 points) Give the waits-for graph for the lock requests in our table. List each edge in the graph in the form:

Ti → Tj with an X label on the arrow indicating Ti is waiting for Tj to release its lock on object X).

(c) (1.5 points) Do we have a deadlock ? Why or why not ?

Explanation:

(a) see table.

(b)

• T1 → T3 because of C

• T2 → T1 because of A

• T3 → T1 because of B

• T3 → T2 because of B

(c) Deadlock exists because there are cycles (T1 -> T3 -> T1) (T1 -> T3 -> T2 -> T1) in the dependency graph.

Q2.5 (Deadlock prevention) (4 points) Consider the following schedule of lock requests (X stands for exclusive, S stands for shared). Assume transactions never release locks in the considered time frame.

Timestep	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**
Transaction 1	X(B)			S(A)				
Transaction 2					X(D)	X(C)		
Transaction 3			S(C)				X(B)	
Transaction 4		X(A)						S(D)
Lock Manager (wait-die)	g	g	g	b	g	b	a	a
Lock Manager (wound-wait)	g	g	g	a	g	a	d	d

(a) (2 points). To prevent deadlocks, the Lock Manager adopts the Wait-Die policy. We assume that in terms of priority: T1 > T2 > T3 > T4. T1 > T2 because T1 is older than T2 (older transactions have higher priority). Write directly in the table, in the wait-die row, ‘g’ whenever the lock request is granted, ‘b’ whenever the lock request is blocked, ‘a’ when the transaction is aborted by the lock manager, and ‘d’ when the transaction is already dead.

(b) (2 points) Under the same priority assumptions of (a), to prevent deadlocks, the Lock Manager now adopts the Wound-Wait policy. Again, write directly in the table, in the wound-wait row, ‘g’ in the bottom row whenever the lock request is granted, ‘b’ whenever the lock request is blocked, ‘a’ when the transaction is granted by causing another one to be aborted by the lock manager, and ‘d’ when the transaction is already dead.

Explanation:

(a) see table

(b) see table

Q2.6 (System R / Selinger query optimization) (12 points) Consider the following schema:

FERRY(ferry_id INTEGER PRIMARY KEY,

from_port INTEGER REFERENCES PORT,

to_port INTEGER REFERENCES PORT,

company_id INTEGER REFERENCES COMPANY);

Ferry statistics: 200 000 records, 100 pages, ferry_id has consecutive values between 1 and 200000.

Ferry indexes:

- (Index1) unclustered B+-tree on company_id, 40 index pages, height 2.
- (Index2) clustered B+-tree index on from_port, 20 index pages, height 2.

PORT(port_id INTEGER PRIMARY KEY,

name VARCHAR2(20),

country VARCHAR2(20),

population INTEGER);

Port statistics: 100000 records, 40 pages, country has 10 possible values, population values are in the range [1 million, 6 million], port_id has consecutive values between 1 and 100000.

Indexes:

- (Index3) clustered B+-tree on population, 20 index pages, height 2
- (Index4) unclustered B+-tree on port_id, 10 index pages, height 2

COMPANY(company_id INTEGER PRIMARY KEY,

name VARCHAR2(20));

Company statistics: 10000 records, 10 pages, company_id has consecutive values between 1 and 10000.

Consider the following query:

```
SELECT *
FROM Ferry F, Port P, Company C
WHERE F.to_port = P.port_id AND F.company_id = C.company_id AND F.company_id >= 4000
      AND P.population > 3 000 000 AND P.Country = 'Singapore'
ORDER BY P.population;
```

Considering each condition in the WHERE clause separately, what is the Reduction Factor (RF)?

(a) (1 point) RF1: P.Country = 'Singapore'

Explanation: 1/10 = 0.1, since there are 10 possible values for country.

(b) (1 point) RF2: F.to_port = P.port_id

Explanation: 1/MAX(100000, 100000) = 1/100000, since there are 100 000 records in the Port table, and port_id is its PK, while to_port is a FK that references the primary key of Port, so there are also at most 100000 distinct values for it.

(c) (1 point) RF3: F.company_id >= 4000

Explanation: Since PKs are between 1 and 10000, we know that the lowest value for company_id (which is PK) is 1, and the highest value is 10000.

So the selectivity is (10000 - 4000)/(10000 - 1 + 1) + 1/10000 = **6001**/10000=0.6

(d) (1 point) RF4: P.population > 3 000 000

Explanation: $(6x10^6) – (3 \times 10^6) / (6 \times 10^6 – 10^6 + 1) = 3 \times 10^6 / (5 \times 10^6 + 1)$ so 16pprox. 0.6

I (1.5 points) Fill all the blanks in the dynamic programming table below for Pass 1 (one-table plans).

Table	Plan	Interesting order plan	I/O cost
Ferry	Filescan	none	100
Ferry	Index1	company_id	200 040 x 0.6 = 120024
Port	Filescan	none	40
Port	Index3	population	60x0.6 = 36
Company	Filescan	none	10

Explanation:

Ferry: The filescan line is clear. We have an access by Index1 with an interesting order on company_id because it can be used as part of a join condition later (e.g., by index nested loop join). Cost: (200,040) * RF3 since the index is unclustered.

Port: The filescan is clear. Interesting order on population, since we have an order by on that. Cost is (20 + 40) xRF4 = 60 x 0.6 = 36 since the index is clustered.
Company: The filescan is clear.

(f) (1 point) Which of these lines should be discarded from this table and why?

Explanation: The filescan on Port is discarded, since a better plan (additionally with an interesting order) is available.

(g) (1.5points) After Pass 2, which of the following plans should be in the dynamic programming table?

(1) (0.5 points) Port [Index3] JOIN Company [Filescan]

(2) (0.5 points) Port [Index3] JOIN Ferry [Index1]

(3) (0.5 points) Ferry [Filescan] JOIN Port [Filescan]

Explanation:
(1) No, because there is no condition joining these two tables, therefore that would be a cross product
(2) Yes, in principle, as we have Port [Index3] from Pass 1 because it has the lowest overall. We have Ferry[Index1] from Pass 1 because it gives an interesting order.
(3) No, Port[Filescan] would not be in the table after Pass 1.

(h) (4 points) Suppose next we need to optimize for queries similar to the one used so far (same structure, but different values for range and equality conditions). What choices could reduce I/O cost of such queries?

(1) (1 point) Change Index3 to be unclustered.

(2) (1 point) Store the Port table as a sorted file on population

(3) (2 points) What else ?

Explanation:
(1) Will not reduce I/O cost, on the contrary, with an unclustered index instead of 1 I/O per matching page of tuples, we would have 1 I/O per matching tuple.

(2) May reduce I/O cost slightly. A sorted file may enable an efficient range lookup for the C.population > … predicate.

(3) Drop the clustered index Index2 on from_port (or make it unclustered), and make Index1 clustered. If the Index1 were clustered, we would have a cost with interesting order of 140 x 0.6 = 84

Q2.7 (Buffer Management) (6 points) We are given a buffer pool with 4 pages, which is empty initially. Answer the following questions given the following access pattern: **A D C B E D A B C A E C**

(a) (1 point) What is the hit rate for MRU?

Explanation: 4/12.

(b) (1 point) By MRU, listed in the order of when they were first placed into the buffer pool, what pages remain in the buffer pool after the given sequence of accesses?

Explanation: D, B, A, C

(c) (1 point) What is the hit rate for clock?

Explanation: 5/12.

(d) (1 point) By clock, listed in the order of when they were first placed into the buffer pool, which pages remain the buffer pool after the given sequence of accesses?

Explanation: A, C, B, E.

(e) (1 point) By clock, which pages have their reference bits set to 1?

Explanation: A, C, E.

(f) (1 point) By clock, which page is the arm of the clock pointing to at the end?

Explanation: A. On a page hit, the arm does not move.

Q2.8 (Files, Pages, Records) (6 points) Consider the following relation:

```
CREATE TABLE dogs (
id INTEGER PRIMARY KEY,
age INTEGER NOT NULL,
weight INTEGER NOT NULL,
good_boy BOOLEAN NOT NULL,
name VARCHAR2(30) NOT NULL,
bread VARCHAR2(10) NOT NULL);
```

INTEGER is 4 bytes long, VARCHAR2(n) can be between 0 (empty string) and n bytes long. BOOLEAN is 1 byte.

(a) (1 point) Looking at records of attributes and their encoding, as the records are of variable size, we will need a record header in the encoding of the record. How big should the record header be? We assume pointers are 2 bytes in size, and that the record header only contains pointers.

Explanation: 4 bytes.
In the record header, we need one pointer for each variable length field. In our schema, these are just the two VARCHAR2 fields. So we need 2 pointers, each 2 bytes.

(b) (1 point) Including the record header, what is the smallest possible record size (in bytes) in our schema?

Explanation: 17 bytes (= 4 + 4 + 4 + 4 + 1+0+0)
4 for the record header, 4 for each of integers, 1 for the Boolean, and 0 for each of the VARCHARs.

(c) (1 point) Including the record header, what is the largest possible record size (in bytes) in our schema?

Explanation: 57 bytes (= 4 + 4 + 4 + 4+1+30+10)

(d) (1 point) Moving to pages of records, suppose we are storing the records using a slotted page layout with variable length records. The page footer contains an integer storing the record count and a pointer to free space, as well as a slot directory storing, for each record, a pointer and length. What is the maximum number of records that we can fit on an 8KB page? (Recall that one KB is 1024 bytes.)

Explanation: 355 records (= (8192 - 4 - 2) / (17 + 4 + 2))

We start with 8192 bytes of space. 4 bytes are used for the record count, and 2 for the pointer to free space. So 8192 - 4 - 2 bytes can use to store records and the slot directory. A record takes at least 17 bytes of space, and for each record we also need to store a slot with a pointer (2 bytes) and a length (4 bytes). So we need 17 + 4 + 2 bytes of space for each record, at least.

(e) (1 point) Suppose we stored the maximum number of records on a page, and then deleted one record. Now we want to insert another record, in which the name is rather short, only 10 bytes in size. Are we guaranteed to be able to do this? Explain why or why not.

Explanation: Yes, since we have some leftover space and the maximal size of the new record is 37 bytes, with the 17 bytes of the deleted record, we can do it.

(f) (1 point) Now suppose we deleted 3 records. Without reorganizing any of the records on the page, we would like to insert another record. Are we guaranteed to be able to do this? Explain why or why not.

Explanation: No; there are 51 free bytes but they may be fragmented - there might not be 57 contiguous bytes.

Q2.9 (Parallel DBMS) (7.5 points) The Singapore karting association decides to create a database keeping all the junior Trainers, Mechanics, and senior Instructors since the creation of the association. The association creates the following relations:

Trainer(name, circuit, trainee)

Instructor(name, circuit, trainee)

Mechanic(name, class, number_repairs)

This data is partitioned across 5 machines, each with B pages of memory in the following manner (note that initially the DB administrator mistakenly assumes a uniform distribution for all values when partitioning):

- Trainer: Hash-partitioned on trainee over machines 1 to 5.
- Instructor: Hash-partitioned on trainee over machines 1 to 5
- Mechanic: Range-partitioned on number_repairs done, over machines 1 to 5

(a) (1.5 points) The DB administrator realizes its mechanics are rather inactive – 70% of them have no repairs (0). We wish to do a parallel scan on this table. Let [M] denote the number of pages in Mechanic and let t denote the time necessary for one I/O. How long will it take for a parallel scan to complete?

Explanation: Because 70% of records will end up on the same machine, the longest time taken for one of the machines to do a scan on its portion of the table will be 0.7 * [M] * t. So this is the slowest node and it will take 0.7 * [M] * t time for the scan to complete

The association knows that many of the instructors used to be trainers, and so they decide to perform a hash-based join on the Instructor and Trainer tables to find who exactly is both a trainer and an instructor still. Assume that we join on the name field, that we have a perfect hash function and that the distribution of names is not skewed in either table. Let [T] be the number of pages in Trainer and [I] be the number of pages in Instructor, with [I] < [T].

(b) (1.5 points) Suppose we first want to remove duplicate entries in Instructor using hashing (grouping). What is the number of passes required to re-partition this table across the 5 machines and de-duplicate its records? We assume that in the shuffle phase, the shuffling of the records is pipelined directly into the hashing operator. Recall each machine has B buffer pages of available memory.

Explanation: Each machine will get [I]/5 pages. If [I]/5 is less than or equal to B, we are done with only 1 pass everywhere. Otherwise, we apply the single-machine hashing formula to a relation with [I]/5 pages to then get an over number of passes of 1 + ceil ($\log_{B-1}$ (ceil ([I]/5B)))

Now assume that the association introduces a new table called Trainee, which tracks young drivers who want to become trainers through practice. Each trainee has one trainer. The schema for this new table is

Trainee (name, age, trainer)

Assume this table is round-robin partitioned across the 5 machines. Now, assume that 30% of all trainees in the table have "Mark" as their trainer. We wish to run the following query:

```
SELECT *
FROM trainee
WHERE trainer = "Mark";
```

(c) (1.5 points) What is the time necessary for the above query to complete, if each machine has B buffer pages and Trainee has [T] pages in total? Let t denote the time taken for 1 I/O.

Explanation: By round-robin partitioning we are only dealing with a single table lookup, we can just run the query locally on each of the 5 machines and concatenate the results. Data skew will not be an issue because round-robin partitioning evenly distributes records across machines. So 1/5 x [T] x t

(d) (1.5 points) If we want to sort Trainee by age, whose values are uniformly distributed, what partitioning scheme would be the best in terms of I/Os?

Explanation: Range partitioning will be the best, since we can individually sort at each machine and concatenate the runs at the end.

(e) (1.5 points) Suppose we have records of all races over the last 10 years:

Race(race_day, race_type, circuit, lap_time, kart_id, driver_name, driver_level)

In order to track and see the evolution of performance (lap_time) of drivers on specific karts, we need to run frequent queries over the latest data (for a specific day), and for specific race types. Which

partitioning scheme and physical design at each node will give the best average latency if we have many analysts querying concurrently this data? Example query:

SELECT avg(lap_time), kart_id, driver_name
FROM Race
WHERE race_day = today – 7 AND race_type='10lap'
GROUP BY kart_id, driver_name;

Explanation: Hash partition on (kart_id, driver_name), in this way query will be spread across all machines, and they will quickly find the answers to be unioned. At each machine a clustered index on (race_time, race_type) in this order since we're looking at "latest" days for specific race types.

Q2.10 (NoSQL - Cassandra) (11 points) Consider the following NoSQL database in Cassandra, on publications and authors thereof. Two tables have already been created by the commands below :

CREATE TABLE Publications (pub_id TEXT, type TEXT, title text, editor text, volume int, isbn text, url text, year int, PRIMARY KEY (pub_id));

CREATE TABLE Authors (pub_id TEXT, author TEXT, position INT, PRIMARY KEY ((author), pub_id));

In the Authors table, the primary key is composed of two attributes, but the data is hash-partitioned by 'author' (the primary key attribute in parenthesis).

Three indexes have also been built:

CREATE INDEX pub_type on publications(type);
CREATE INDEX authors_pub_id on authors(pub_id);
CREATE INDEX authors_position on authors(position);

Give the CQL queries for the questions below. For each question, take and describe all the necessary steps in order to have a query that can be executed (instead of one that raises an error message). Anything is allowed (creating indexes, using the "Alow filtering" option, creating new tables, creating new data types, etc).

(a) (1 point) List of 10 (any) publication titles.

(b) (1 point) Title and editor of the publication with id “Pub_12345”.

(c) (1 point) Number of publications whose type is “Book”.

(d) (1 point) Number of publications whose title is equal to “Survey”.

(e) (1 point) Number of publications whose type is “Article” and title is “Survey”.

(f) (1 point) Number of authors whose position is equal to 1.

(g) (1 point) The authors of the publication with id “Pub_12345”.

(h) (2 point) The authors of the publication(s) whose title is “Survey”.

(j) (2 point) The titles and position of publications authored by “John Smith”.

Explanation:

(a) SELECT title FROM publications LIMIT 10;

(b) SELECT title, editor FROM publications WHERE pub_id='Pub_12345';

(c) SELECT COUNT(*) FROM publications WHERE pub_type='Book'; this query is possible since we have an index on type

(d) SELECT COUNT(*) FROM publications WHERE title='Survey' ALLOW FILTERING; we need the ALLOW FILTERING option since the lookup condition is not on the partition key and we do not have an index either on the title. The better approach would be to create a secondary index on title, and then execute without "ALLOW FILTERING".

CREATE INDEX pub_title on publications(title);

SELECT COUNT(*) FROM publications WHERE title='Survey';

(e) SELECT COUNT(*) FROM publications WHERE title='Survey' and type='Article' ALLOW FILTERING; ALLOW FILTERING is necessary since we cannot use the two indexes jointly.

(f) SELECT COUNT(*) FROM authors WHERE position=1; We have an index on position, so this works

(g) SELECT author FROM authors WHERE pub_id='Pub_12345'; we have an index on pub_id so this works.

(h) Here we need to join between tables publications and authors. There is no way to do a join in CQL. So a first denormalization step has been done on this dataset with some table called "authors_publications".

CREATE TABLE authors_publications (

pub_id TEXT, **author TEXT**, type TEXT, title text, editor text, volume int, isbn text, url text, year int, **position int**, PRIMARY KEY ((author), pub_id));

CREATE INDEX authors_publications_title on authors_publications(title);

SELECT author FROM authors_publications WHERE title='Survey';

(i) SELECT title, position FROM authors_publications WHERE author="John Smith";

END OF PAPER